📄 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the **Resume Screening System**, an NLP-based web application that automatically evaluates and ranks resumes based on their relevance to a given job description using **TF-IDF + Cosine Similarity** and **BERT** models.

This system aims to reduce manual effort, save time, and improve accuracy in the recruitment process.

1.2 Scope

The Resume Screening System will:

- Accept multiple resumes in PDF format
- Accept a job description as input
- Extract and preprocess resume text
- Compute similarity scores using NLP techniques
- Rank resumes based on relevance
- Display results through a web interface

The system is developed using **Python Flask** and deployed on **Render**.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
BERT	Bidirectional Encoder Representations from Transformers
JD	Job Description
API	Application Programming Interface

1.4 Technologies Used

- Python
- Flask
- Scikit-learn

- Sentence Transformers (BERT)
- HTML, CSS
- Render (Cloud Deployment)

---

2. Overall Description

2.1 Product Perspective

The Resume Screening System is a **standalone web-based application** that automates resume screening using machine learning and NLP algorithms.

2.2 Product Features

- Upload multiple resumes (PDF)
- Input job description text
- Automatic text preprocessing
- Resume-to-JD similarity matching
- Resume ranking
- Web-based result display

2.3 User Classes and Characteristics

User	Description
Recruiter / HR	Screens resumes using job description
Admin	Manages deployment and system configuration

2.4 Operating Environment

- Web Browser (Chrome, Edge, Firefox)
- Server: Render Cloud Platform
- OS: Windows / Linux
- Python 3.9+

2.5 Design and Implementation Constraints

- Internet connection required
- Limited file size for resume uploads

- Processing time depends on resume count

2.6 Assumptions and Dependencies

- Resumes are in readable PDF format
- Job description is provided in English
- Server resources are sufficient for NLP processing

---

3. System Architecture

3.1 Architecture Overview

User Interface (HTML/CSS)

↓

Flask Backend

↓

NLP Processing Module

↓

Similarity Computation

↓

Ranking & Results

3.2 Modules Description

a) Resume Upload Module

- Accepts multiple PDF resumes
- Stores resumes temporarily for processing

b) Text Extraction Module

- Extracts text from PDF resumes using Python libraries

c) Preprocessing Module

- Lowercasing
- Stopword removal
- Tokenization
- Lemmatization

d) NLP Similarity Module

- TF-IDF + Cosine Similarity
- BERT-based semantic similarity (optional)

e) Ranking Module

- Sorts resumes based on similarity score

f) Web Interface Module

- Displays ranked resumes with similarity scores

---

4. Functional Requirements

4.1 Resume Upload

- The system shall allow users to upload multiple resumes.
- The system shall validate file format (PDF only).

4.2 Job Description Input

- The system shall allow users to input job description text.

4.3 Text Processing

- The system shall clean and preprocess resume and JD text.
- The system shall remove stopwords and irrelevant characters.

4.4 Similarity Calculation

- The system shall compute similarity using TF-IDF + Cosine Similarity.
- The system may compute similarity using BERT embeddings.

4.5 Resume Ranking

- The system shall rank resumes based on similarity score.
- The system shall display results in descending order.

---

5. Non-Functional Requirements

5.1 Performance

- The system should process resumes within reasonable time.
- The system should support at least 10 resumes per request.

5.2 Security

- Uploaded resumes should not be stored permanently.
- No sensitive data should be shared externally.

5.3 Usability

- Simple and intuitive user interface.
- Minimal technical knowledge required.

5.4 Scalability

- The system should be deployable on cloud platforms.
- Can be extended for large datasets in future.

5.5 Reliability

- The system should handle invalid files gracefully.
- The system should provide accurate ranking results.

---

6. External Interface Requirements

6.1 User Interface

- Web-based UI using HTML and CSS
- Upload form and results table

6.2 Hardware Interface

- No special hardware required

6.3 Software Interface

- Flask Web Framework
- Python NLP Libraries

---

7. Deployment Requirements

7.1 Deployment Platform

- Render Cloud Platform

7.2 Deployment Steps

1. Create Flask application
2. Add requirements.txt
3. Push code to GitHub
4. Connect repository to Render
5. Deploy web service

---

8. Future Enhancements

- Skill extraction and keyword highlighting
- Resume classification by domain
- Integration with ATS systems
- Support for DOCX resumes
- Database integration

---

9. Conclusion

The Resume Screening System provides an efficient, automated, and intelligent approach to resume evaluation using NLP techniques. By leveraging TF-IDF, Cosine Similarity, and BERT models, the system improves recruitment efficiency and accuracy while reducing manual effort.